



Bahria University, Islamabad
Department of Software Engineering

Numerical Analysis Lab
(Fall-2026)

Teacher: Engr. Sadaf Farhan

Student Name: Faisal khan

Enrollment: 01-131232-105

Lab Journal: 04

Date: 24-09-2026

Task No:	Task Wise Marks		Documentation Marks		Total Marks (20)
	Assigned	Obtained	Assigned	Obtained	
1	3		5		
2	3				
3	3				
4	3				
5	3				

Comments:

Signature

Understand the concept and importance of difference operators in numerical analysis.

WORKED EXAMPLE · 3

The Difference Table

x	f(x)	1st	2nd	3rd	4th	5th
-2	15					
		-14				
-1	1		14			
		0		-12		
0	1		2		24	
		2		12		0
1	3		14		24	
		16		36		0
2	19		50		24	
		66		60		
3	85		110			
		176				
4	261					

The 4th differences are all 24 and the 5th differences are 0 — the data behind this table is a polynomial of degree 4, so its 5th difference must vanish.

Code:

```
import math

def ordinal_suffix(n):
    """Returns ordinal string (1st, 2nd, 3rd, 4th, etc.)."""
    if 11 <= (n % 100) <= 13:
        return f"{n}th"
    suffixes = {1: "st", 2: "nd", 3: "rd"}
    return f"{n}{suffixes.get(n % 10, 'th')}"

def run_lab4_complete():
    print("=====")
    print("      NUMERICAL METHODS - LAB 4 PROGRAM      ")
    print("      Difference Operators & Lookup Tool      ")
    print("=====\\n")

    # STEP 1: Get total number of data points
    try:
        n = int(input("Enter number of data points (n): ").strip())
        if n < 2:
            print("Error: You need at least 2 points to build a table.")
            return
    except ValueError:
        print("Invalid input. Please enter an integer.")
        return

    # STEP 2: Take x and f(x) input step-by-step
    x_vals = []
```

```
y_vals = []

print("\nEnter data points one by one:")
for i in range(n):
    print(f"\n--- Point {i + 1} ---")
    while True:
        try:
            x_val = float(input("  x  : ").strip())
            y_val = float(input("  f(x): ").strip())
            x_vals.append(x_val)
            y_vals.append(y_val)
            break
        except ValueError:
            print("Invalid input! Please enter numbers only.")

# STEP 3: Construct Difference Matrix
table = [[0.0 for _ in range(n)] for _ in range(n)]
for i in range(n):
    table[0][i] = y_vals[i]

for j in range(1, n):
    for i in range(n - j):
        table[j][i] = table[j - 1][i + 1] - table[j - 1][i]

# STEP 4: Print Staggered Difference Table
col_width = 12
headers = [f"{'x':^{col_width}}", f"{'f(x)':^{col_width}}"]
for j in range(1, n):
    headers.append(f"{'ordinal_suffix(j):^{col_width}}")

print("\n" + "=" * (col_width * (n + 1)))
print("".join(headers))
print("=" * (col_width * (n + 1)))

for i in range(n):
    x_str = (
        f"{int(x_vals[i])}"
        if x_vals[i].is_integer()
        else f"{x_vals[i]:.2f}"
    )
    y_str = (
        f"{int(y_vals[i])}"
        if y_vals[i].is_integer()
        else f"{y_vals[i]:.2f}"
    )

    row_str = f"{x_str:^{col_width}}{y_str:^{col_width}}"

# Even order differences
for j in range(2, n, 2):
```

```

        if i < n - j:
            val = table[j][i]
            v_str = f"{int(val)}" if val.is_integer() else f"{val:.2f}"
            row_str += " " * col_width + f"{v_str:^{col_width}}"

    print(row_str)

    # Odd order differences (staggered row)
    if i < n - 1:
        stag_str = " " * (col_width * 2)
        for j in range(1, n, 2):
            if i < n - j:
                val = table[j][i]
                v_str = f"{int(val)}" if val.is_integer() else
f"{val:.2f}"
                stag_str += f"{v_str:^{col_width}}" + " " * col_width
        print(stag_str)

    print("=" * (col_width * (n + 1)))

    # STEP 5: Polynomial Vanishing Rule Analysis
    poly_degree = None
    for j in range(1, n):
        diffs = [table[j][i] for i in range(n - j)]
        if all(abs(d) < 1e-9 for d in diffs) and j > 1:
            poly_degree = j - 1
            const_val = table[poly_degree][0]
            c_str = (
                f"{int(const_val)}"
                if const_val.is_integer()
                else f"{const_val:.2f}"
            )
            print(
                f"\n[ANALYSIS]: The {ordinal_suffix(poly_degree)} differences
are all {c_str} "
                f"and the {ordinal_suffix(j)} differences are 0 - "
                f"the data behind this table is a polynomial of degree
{poly_degree}, "
                f"so its {ordinal_suffix(j)} difference must vanish.\n"
            )
            break

    # STEP 6: Origin Point Selection & Subscript Relabeling
    print("-" * 50)
    print("ORIGIN POINT SELECTION")
    print("-" * 50)
    print("Available x values:")
    for idx, xv in enumerate(x_vals):
        x_s = f"{int(xv)}" if xv.is_integer() else f"{xv}"
        print(f"  Index {idx + 1}: x = {x_s}")

```

```
try:
    origin_choice = int(
        input(f"\nSelect origin point index (1 to {n}): ").strip()
    )
    origin_idx = origin_choice - 1
    if not (0 <= origin_idx < n):
        print("Invalid index. Defaulting to index 1.")
        origin_idx = 0
except ValueError:
    print("Invalid input. Defaulting to index 1.")
    origin_idx = 0

x0 = x_vals[origin_idx]
y0 = y_vals[origin_idx]
x0_str = f"{int(x0)}" if x0.is_integer() else f"{x0}"
y0_str = f"{int(y0)}" if y0.is_integer() else f"{y0}"

print(f"\nOrigin Selected: x0 = {x0_str} (f0 = {y0_str})\n")

# Build database of forward, backward, and central labels
labels_db = []
for k in range(1, n):
    for i in range(n - k):
        val = table[k][i]

        # Forward:  $\Delta^k f_r$  where  $r = i - \text{origin\_idx}$ 
        r_fwd = i - origin_idx
        fwd_label = f"A^{k}f{r_fwd}" if k > 1 else f"Af{r_fwd}"

        # Backward:  $\nabla^k f_r$  where  $r = i + k - \text{origin\_idx}$ 
        r_bwd = i + k - origin_idx
        bwd_label = f"V^{k}f{r_bwd}" if k > 1 else f"Vf{r_bwd}"

        # Central:  $\delta^k f_r$ 
        c_val = i + k / 2.0 - origin_idx
        if c_val % 1 == 0:
            cnt_label = (
                f"6^{k}f{int(c_val)}" if k > 1 else f"6f{int(c_val)}"
            )
        else:
            num = int(round(c_val * 2))
            cnt_label = f"6^{k}f{num}/2" if k > 1 else f"6f{num}/2"

        labels_db.append(
            {
                "col": k,
                "row": i,
                "val": val,
                "fwd": fwd_label,
```

```
        "bwd": bwd_label,
        "cnt": cnt_label,
        "r_fwd": r_fwd,
    }
)

# STEP 7: Display Evaluation at Origin
print("=" * 50)
print(" VALUES RELATIVE TO ORIGIN")
print("=" * 50)

print("\nForward differences from origin (A):")
for item in labels_db:
    if item["row"] == origin_idx:
        v = item["val"]
        v_str = f"{int(v)}" if v.is_integer() else f"{v:.2f}"
        print(f"    {item['fwd']:<10} = {v_str}")

print("\nBackward differences at origin (V):")
for item in labels_db:
    if item["row"] + item["col"] == origin_idx:
        v = item["val"]
        v_str = f"{int(v)}" if v.is_integer() else f"{v:.2f}"
        print(f"    {item['bwd']:<10} = {v_str}")

print("\nCentral differences around origin (6):")
for item in labels_db:
    c_val = item["row"] + item["col"] / 2.0 - origin_idx
    if abs(c_val) < 1e-5:
        v = item["val"]
        v_str = f"{int(v)}" if v.is_integer() else f"{v:.2f}"
        print(f"    {item['cnt']:<10} = {v_str}")

# STEP 8: Interactive Query Mode
print("\n" + "=" * 50)
print(" INTERACTIVE QUERY MODE")
print("=" * 50)
print("Examples:")
print("  - Type an operator label: Af0, Vf1, 6f1/2, A^2f1, V^3f2")
print("  - Type a number: 16, 24, 60 (to find its equivalent labels)")
print("  - Type 'q' to quit.\n")

while True:
    query = input("query> ").strip()
    if query.lower() == "q":
        break
    if not query:
        continue

    # Check if query is a number
```

```
try:
    num = float(query)
    matches = []
    for item in labels_db:
        if abs(item["val"] - num) < 1e-5:
            matches.extend([item["fwd"], item["bwd"], item["cnt"]])

    if matches:
        print(
            f"Value {int(num) if num.is_integer() else num} matches:
{'', '.join(matches)}"
        )
    else:
        print(f"Value {query} not found in the difference table.")
except ValueError:
    # Search by label
    found = False
    for item in labels_db:
        if query in (item["fwd"], item["bwd"], item["cnt"]):
            v = item["val"]
            v_str = f"{int(v)}" if v.is_integer() else f"{v:.2f}"
            fwd_equiv = item["fwd"]
            print(
                f"Found: {query} = {v_str} (Forward equivalent:
{fwd_equiv})"
            )
            found = True
            break
    if not found:
        print(
            f"Label '{query}' not found. Check formatting (e.g., Af0,
Vf1, 6f1/2)."
        )

    print("\nProgram finished. Excellent work!")

if __name__ == "__main__":
    run_lab4_complete()
```

Output:

```
=====
NUMERICAL METHODS - LAB 4 PROGRAM
Difference Operators & Lookup Tool
=====

Enter number of data points (n): 7

Enter data points one by one:

--- Point 1 ---
x   : -2
f(x): 15

--- Point 2 ---
x   : -1
f(x): 1

--- Point 3 ---
x   : 0
f(x): 1

--- Point 4 ---
x   : 1
f(x): 3

--- Point 5 ---
x   : 2
f(x): 19

--- Point 6 ---
x   : 3
f(x): 85

--- Point 7 ---
x   : 4
f(x): 2661

=====
x      f(x)    1st    2nd    3rd    4th    5th    6th
=====
-2      15         -14     14     -12     24         0      2400
-1       1          0      2      12     24        2400
0        1          2     14     36    2424
1         3         16     50    2460
2        19        66    2510
3        85       2576
4       2661
=====
```

```
-----
ORIGIN POINT SELECTION
-----

Available x values:
Index 1: x = -2
Index 2: x = -1
Index 3: x = 0
Index 4: x = 1
Index 5: x = 2
Index 6: x = 3
Index 7: x = 4

Select origin point index (1 to 7): 4

Origin Selected: x0 = 1 (f0 = 3)

=====
VALUES RELATIVE TO ORIGIN
=====

Forward differences from origin (A):
Af0      = 16
A^2f0    = 50
A^3f0    = 2460

Backward differences at origin (V):
Vf0      = 2
V^2f0    = 2
V^3f0    = -12

Central differences around origin (6):
6^2f0    = 14
6^4f0    = 24
6^6f0    = 2400

=====
INTERACTIVE QUERY MODE
=====

Examples:
- Type an operator label: Af0, Vf1, 6f1/2, A^2f1, V^3f2
- Type a number: 16, 24, 60 (to find its equivalent labels)
- Type 'q' to quit.

query> 24
Value 24 matches: A^4f-3, V^4f1, 6^4f-1, A^4f-2, V^4f2, 6^4f0
query> q

Program finished. Excellent work!

C:\Users\hp\Desktop\NA-lab 03>
```

Thank you